

D. TUGAS/LATIHAN SOAL

Deskripsi Tugas: Anda diminta untuk mengimplementasikan algoritma KMeans pada dataset yang telah dipilih dan mengevaluasi hasil clustering menggunakan Metode Elbow dan Metode Silhouette. Mall Customers Dataset (<https://www.kaggle.com/vichoudhary7/customer-segmentation-tutorial-in-python>)

1. Praproses Data:

- Lakukan eksplorasi data dan visualisasi awal untuk memahami struktur dataset (misalnya, dengan menggunakan grafik distribusi dan box plot).
- Pastikan data sudah bersih dan siap untuk clustering (misalnya, menangani missing values jika ada).

2. Implementasi KMeans:

- Tentukan jumlah cluster yang optimal menggunakan **Metode Elbow** dengan menghitung inertia untuk berbagai nilai kkk (jumlah cluster).
- Hitung **Silhouette Score** untuk setiap nilai kkk dan plot grafiknya untuk melihat kualitas clustering.

3. Evaluasi Clustering:

- Tentukan jumlah cluster optimal berdasarkan hasil dari **Metode Elbow** dan **Metode Silhouette**.
- Implementasikan KMeans dengan jumlah cluster yang telah ditentukan dan analisis hasil clustering.

4. Visualisasi Hasil Clustering:

- Gunakan **scatter plot** atau visualisasi 2D lainnya untuk memetakan hasil clustering pada dataset.
- Warnai titik data berdasarkan cluster yang ditentukan oleh KMeans.

5. Interpretasi Hasil:

- Jelaskan hasil clustering Anda, termasuk pemisahan antar cluster yang terlihat.
- Berdasarkan Silhouette Score dan Inertia, apakah hasil clustering sudah optimal? Jelaskan.

PERTEMUAN KE-11: KNN (K-NEAREST NEIGBORS)

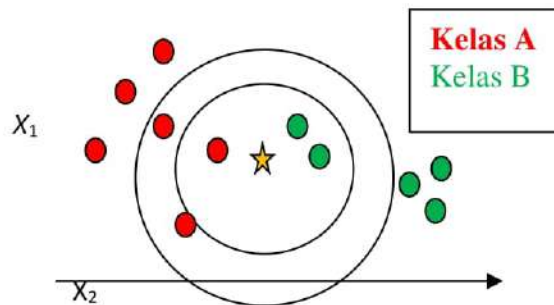
A. TUJUAN

Setelah menempuh praktikum ini diharapkan mahasiswa mampu

- memahami cara kerja dari metode KNN (K-Nearest Neighbors).
- mampu menerapkan metode KNN (K-Nearest Neighbors) dengan bahasa pemrograman Python dan menggunakannya untuk menyelesaikan masalah klasifikasi.

B. MATERI

Algoritma *K-Nearest Neighbor* (K-NN) adalah sebuah metode klasifikasi terhadap sekumpulan data berdasarkan pembelajaran data yang sudah terklasifikasi sebelumnya. Termasuk dalam supervised learning, dimana hasil query instance yang baru diklasifikasikan berdasarkan mayoritas kedekatan jarak dari kategori yang ada dalam K-NN.



Gambar 11.1. Ilustrasi KNN

Gambar 11.1 menunjukkan bahwa data baru (bintang berwarna kuning) akan dikelompokkan ke kelas B jika $k = 3$ dan masuk ke kelas jika diambil $k = 5$. Dekat atau jauhnya tetangga biasanya dihitung berdasarkan *Euclidean Distance*, atau dapat juga menggunakan rumus jarak yang lain. Kedekatan dapat dianggap sebagai invers jarak, alias berbanding terbalik dengan jarak. Semakin kecil jarak antara dua instance, semakin besar “kedekatan” antara dua instance tersebut. Dengan demikian, k nearest neighbours dari sebuah instance x didefinisikan sebagai k instance yang memiliki jarak terkecil (kedekatan terbesar, nearest) dengan x .

Langkah Algoritma K-NN adalah sebagai berikut.

Misalkan kita memiliki:

- **Himpunan data training:**
 $D = \{(x_1, Y_1), (x_2, Y_2), \dots, (x_n, Y_n)\}$
 dengan x_i adalah fitur input dan Y_i adalah label kelas.
- **Data**
 x_q yang akan dklasifikasikan.

uji:

1. Menentukan parameter k

k adalah jumlah tetangga terdekat yang akan digunakan untuk prediksi. Pemilihan k dapat dilakukan berdasarkan eksperimen atau metode seperti cross-validation.

2. Menghitung jarak Euclidean antara data uji dan seluruh data training

Jarak Euclidean antara dua titik dalam ruang dimensi m didefinisikan sebagai:

$$d(x_q, x_i) = \sqrt{\sum_{j=1}^m (x_{qj} - x_{ij})^2}$$

dengan:

- $x_q = (x_{q1}, x_{q2}, \dots, x_{qm})$ sebagai vektor fitur data uji
- $x_i = (x_{i1}, x_{i2}, \dots, x_{im})$ sebagai vektor fitur data training ke- i

Jarak ini dihitung untuk setiap $i \in \{1, 2, \dots, n\}$.

3. Mengurutkan hasil perhitungan jarak secara ascending

Setelah mendapatkan jarak $d(x_q, x_i)$ untuk setiap data training x_i , kita mengurutkan nilai jarak tersebut dari yang terkecil hingga terbesar.

4. Mengumpulkan kategori Y dari k tetangga terdekat

Dari hasil pengurutan, kita ambil k data training dengan jarak terkecil. Kategorinya membentuk himpunan baru:

$$N_k = \{Y_{i1}, Y_{i2}, \dots, Y_{ik}\}$$

dengan Y_{ij} adalah kelas dari kkk tetangga terdekat.

5. Menentukan kategori mayoritas sebagai hasil prediksi

Kelas dari data uji x_q ditentukan berdasarkan kategori yang paling sering muncul dalam himpunan N_k :

$$Y_q = \text{mode}(N_k)$$

di mana $\text{mode}()$ adalah nilai yang paling sering muncul.

Kelebihan dari K-NN adalah

1. Sangat nonlinear. kNN merupakan salah satu algoritma (model) pembelajaran mesin yang bersifat nonparametrik. Pembahasan mengenai model parametrik dan model non parametrik bisa menjadi artikel sendiri, namun secara singkat, definisi model nonparametrik adalah model yang

tidak mengasumsikan apa-apa mengenai distribusi instance di dalam dataset. Model nonparametrik biasanya lebih sulit diinterpretasikan, namun salah satu kelebihanannya adalah garis keputusan kelas yang dihasilkan model tersebut bisa jadi sangat fleksibel dan nonlinear.

2. Mudah dipahami dan diimplementasikan. Dari paparan yang diberikan dan penjelasan cara menghitung jarak dalam artikel ini, cukup jelas bahwa algoritma kNN mudah dipahami dan juga mudah diimplementasikan. Untuk mengklasifikasi instance x menggunakan kNN, kita cukup mendefinisikan fungsi untuk menghitung jarak antar-instance, menghitung jarak x dengan semua instance lainnya berdasarkan fungsi tersebut, dan menentukan kelas x sebagai kelas yang paling banyak muncul dalam k instance terdekat.

Kekurangan K-NN adalah

1. Perlu menunjukkan parameter K (jumlah tetangga terdekat)
2. Tidak menangani nilai hilang (missing value) secara implisit
3. Sensitif terhadap data pencilan (outlier)
4. Rentan terhadap variabel yang non-informatif
5. Rentan terhadap dimensionalitas yang tinggi
6. Rentan terhadap perbedaan rentang variabel
7. Nilai komputasi yang tinggi.

Contoh 11.1

Misalkan diketahui 6 data training berikut:

ID	x_{i1}	x_{i2}	Kelas Y
1	2	4	A
2	4	6	B
3	5	8	B
4	3	7	A
5	6	5	B
6	7	9	A

Gunakan metode KKN untuk mengklasifikasikan **data uji**

$x_q = (4,5)$ dengan $k = 5$.

Jawaban

1. Hitung Jarak Euclidean

Menggunakan rumus jarak Euclidean:

$$d(x_q, x_i) = \sqrt{(x_{q1} - x_{i1})^2 + (x_{q2} - x_{i2})^2}$$

Hitung jarak untuk setiap data training:

- a. $d(x_q, x_1) = \sqrt{(4 - 2)^2 + (5 - 4)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.24$
- b. $d(x_q, x_2) = \sqrt{(4 - 4)^2 + (5 - 6)^2} = \sqrt{0 + 1} = \sqrt{1} = 1.00$
- c. $d(x_q, x_3) = \sqrt{(4 - 5)^2 + (5 - 8)^2} = \sqrt{1 + 9} = \sqrt{10} \approx 3.16$
- d. $d(x_q, x_4) = \sqrt{(4 - 3)^2 + (5 - 7)^2} = \sqrt{1 + 4} = \sqrt{5} \approx 2.24$
- e. $d(x_q, x_5) = \sqrt{(4 - 6)^2 + (5 - 5)^2} = \sqrt{4 + 0} = \sqrt{4} = 2.00$
- f. $d(x_q, x_6) = \sqrt{(4 - 7)^2 + (5 - 9)^2} = \sqrt{9 + 16} = \sqrt{25} = 5.00$

2. Urutkan Jarak Secara Ascending

Dari hasil perhitungan jarak:

ID	x_{i1}	x_{i2}	Kelas Y	Jarak $d(x_q, x_i)$
2	4	6	B	1.00
5	6	5	B	2.00
1	2	4	A	2.24
4	3	7	A	2.24
3	5	8	B	3.16
6	7	9	A	5.00

3. Ambil $k = 5$ Data Terdekat

5 data diambil dengan jarak terkecil:

ID Kelas YY

2 B

5 B

1 A

4 A

3 B

5. Tentukan Kelas Mayoritas

- B (3 kali) → ID 2, 5, 3
- A (2 kali) → ID 1, 4

Mayoritas adalah **kelas B**, sehingga prediksi untuk $x_q = (4,5)$ adalah:

$$Y_q = B$$

Kesimpulan

Berdasarkan algoritma **KNN** dengan $k = 5$, data uji $x_q = (4,5)$ diklasifikasikan sebagai **kelas B**.

Contoh 11.2

Buatlah program metode K-NN untuk menyelesaikan klasifikasi data diabetes.

Jawaban**Kode Program 11.1. Program KNN**

No	Kode Program
1	import random
2	import math
3	import pandas as pd
4	import matplotlib.pyplot as plt
5	from sklearn.metrics import accuracy_score, precision_score,
6	recall_score, f1_score
7	from sklearn.model_selection import train_test_split
8	from sklearn.preprocessing import MinMaxScaler
9	def euclidean_distance(point1, point2):
10	distance = 0
11	for x, y in zip(point1, point2):
12	distance += (x - y) ** 2
13	return math.sqrt(distance)
14	def bubble_sort(arr):
15	n = len(arr)
16	for i in range(n - 1):
17	for j in range(n - i - 1):
18	if arr[j][0] > arr[j + 1][0]:
19	arr[j], arr[j + 1] = arr[j + 1], arr[j]
20	def knn(train_data, train_labels, test_point, k):
21	distances = []
22	for i in range(len(train_data)):
23	dist = euclidean_distance(test_point, train_data[i])
24	distances.append((dist, train_labels[i]))
25	bubble_sort(distances) # Sort distances using bubble sort
26	k_nearest_labels = []
27	for i in range(k):
28	k_nearest_labels.append(distances[i][1])
29	
40	label_counts = {}
41	for label in k_nearest_labels:
42	if label in label_counts:
43	label_counts[label] += 1
44	else:
45	label_counts[label] = 1
46	return max(label_counts, key=label_counts.get) # Majority vote
47	# Membaca dataset (gantikan dengan path dataset Anda jika diperlukan)
48	

```

49 data = pd.read_csv('diabetes.csv') # Dataset Anda, pastikan file
50 tersedia
51 # Memisahkan fitur dan target
52 X = data[['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
53 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']].values
54 y = data['Outcome'].values
55 # Normalize data using Min-Max Scaling
56 scaler = MinMaxScaler()
57 normalized_data = scaler.fit_transform(X)
58 # Split dataset into training and testing
59 train_data, test_data, train_labels, test_labels =
60 train_test_split(normalized_data, y, test_size=0.2, random_state=42)
61 # Run KNN classification for test data
62 k = 50 # Number of nearest neighbors
63 predictions = []
64 for test_point in test_data:
65     predictions.append(knn(train_data, train_labels, test_point, k))
66 # Calculate metrics
67 accuracy = accuracy_score(test_labels, predictions)
68 precision = precision_score(test_labels, predictions)
69 recall = recall_score(test_labels, predictions)
70 f1 = f1_score(test_labels, predictions)
71 # Print results
72 print("Actual labels:", test_labels)
73 print("Predicted labels:", predictions)
74 print(f"Accuracy: {accuracy:.2f}")
75 print(f"Precision: {precision:.2f}")
76 print(f"Recall: {recall:.2f}")
77 print(f"F1 Score: {f1:.2f}")
78 # Plot metrics comparison
79 metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
80 values = [accuracy, precision, recall, f1]
81 plt.figure(figsize=(8, 5))
82 plt.bar(metrics, values, color=['blue', 'green', 'red', 'purple'])
83 plt.ylim(0, 1)
84 plt.xlabel("Metrics")
85 plt.ylabel("Score")
86 plt.title("Performance Metrics of KNN Model")
plt.show()

```

Keluaran dari program 11.1 adalah

Actual labels: [0000000000101001001100000100101111011
1010001011000011100000110010001010001
0000001100011100000100001000101001110
0010010000010001110110110111000000010
010010]

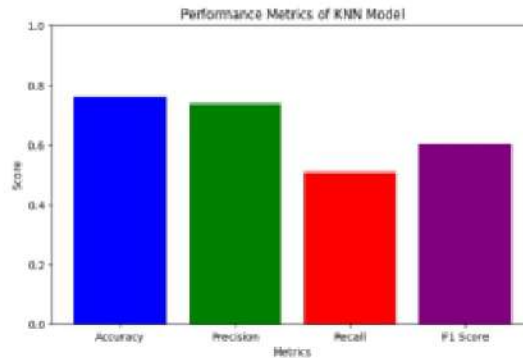
Predicted labels: [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]

Accuracy: 0.76

Precision: 0.74

Recall: 0.51

F1 Score: 0.60



Gambar 11.2 Hasil Program pada Contoh 11.2.

Contoh 11.3

Buatlah program metode K-NN untuk menyelesaikan klasifikasi data diabetes dengan paket program dan tentukan k terbaiknya antara 1 sampai 50.

Jawaban**Kode Program 11.2.** Program KNN dengan paket program

No	Kode Program
1	import pandas as pd
2	from sklearn.model_selection import train_test_split
3	from sklearn.preprocessing import MinMaxScaler
4	from sklearn.neighbors import KNeighborsClassifier
5	from sklearn.metrics import accuracy_score, precision_score,
6	recall_score, f1_score
7	import matplotlib.pyplot as plt
8	# Membaca dataset (gantilah dengan path dataset Anda jika diperlukan)
9	data = pd.read_csv('diabetes.csv') # Dataset Anda, pastikan file
10	tersedia
11	# Memisahkan fitur dan target
12	X = data[['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
13	'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']].values
14	y = data['Outcome'].values
15	# Normalize data using Min-Max Scaling
16	scaler = MinMaxScaler()
17	normalized_data = scaler.fit_transform(X)
18	# Split dataset into training and testing
19	train_data, test_data, train_labels, test_labels =
20	train_test_split(normalized_data, y, test_size=0.2, random_state=42)
21	# Menentukan rentang nilai k untuk eksperimen
22	k_values = range(1, 50) # Bisa juga mencoba lebih banyak nilai k
23	accuracies = []
24	# Mencoba berbagai nilai k
25	for k in k_values:
26	# Membuat dan melatih model KNN
27	knn = KNeighborsClassifier(n_neighbors=k)
28	knn.fit(train_data, train_labels)
29	
30	# Melakukan prediksi
31	predictions = knn.predict(test_data)
32	
33	# Hitung akurasi untuk nilai k tersebut
34	accuracy = accuracy_score(test_labels, predictions)
35	accuracies.append(accuracy)
36	
37	# Plot hubungan antara k dan akurasi
38	plt.figure(figsize=(8, 5))
39	plt.plot(k_values, accuracies, marker='o', color='b', linestyle='-',
40	linewidth=2)
41	plt.xlabel('Nilai k')
42	plt.ylabel('Akurasi')
43	plt.title('Hubungan antara Nilai k dan Akurasi pada Model KNN')
44	plt.grid(True)

```

55 # Menampilkan label sumbu X sebagai kelipatan 2
56 plt.xticks(range(1, 50, 2)) # Menampilkan kelipatan 2
57 plt.show()
58 # Pilih nilai k terbaik berdasarkan akurasi tertinggi
59 best_k = k_values[accuracies.index(max(accuracies))]
60 print(f"Nilai k terbaik berdasarkan akurasi: {best_k}")
61 # Membuat model KNN dengan k terbaik
62 knn_best = KNeighborsClassifier(n_neighbors=best_k)
63 knn_best.fit(train_data, train_labels)
64 # Melakukan prediksi untuk k terbaik
65 best_predictions = knn_best.predict(test_data)
66 # Menghitung metrik lainnya
67 precision = precision_score(test_labels, best_predictions)
68 recall = recall_score(test_labels, best_predictions)
69 f1 = f1_score(test_labels, best_predictions)
70 ac = accuracy_score(test_labels, best_predictions)
71 # Menampilkan hasil
72 print(f"Precision untuk k={best_k}: {precision:.2f}")
73 print(f"Recall untuk k={best_k}: {recall:.2f}")
74 print(f"F1 Score untuk k={best_k}: {f1:.2f}")
75 print(f"Aakurasi Score untuk k={best_k}: {ac:.2f}")

```

Keluaran Kode Program 11.2 adalah

Nilai k terbaik berdasarkan akurasi: 46

Precision untuk k=46: 0.78

Recall untuk k=46: 0.51

F1 Score untuk k=46: 0.62

Aakurasi Score untuk k=46: 0.77



Gambar 11.3. Hubungan antara K dan Akurasi

C. TUGAS/LATIHAN SOAL

1. Lakukan percobaan yang sama dengan contoh 11.3 untuk data yang sama tetapi sudah melalui pemeriksaan outlier / data yang tidak logis dan penanganannya.
2. Modifikasi Kode Program 11.2 untuk menyelesaikan permasalahan regresi.